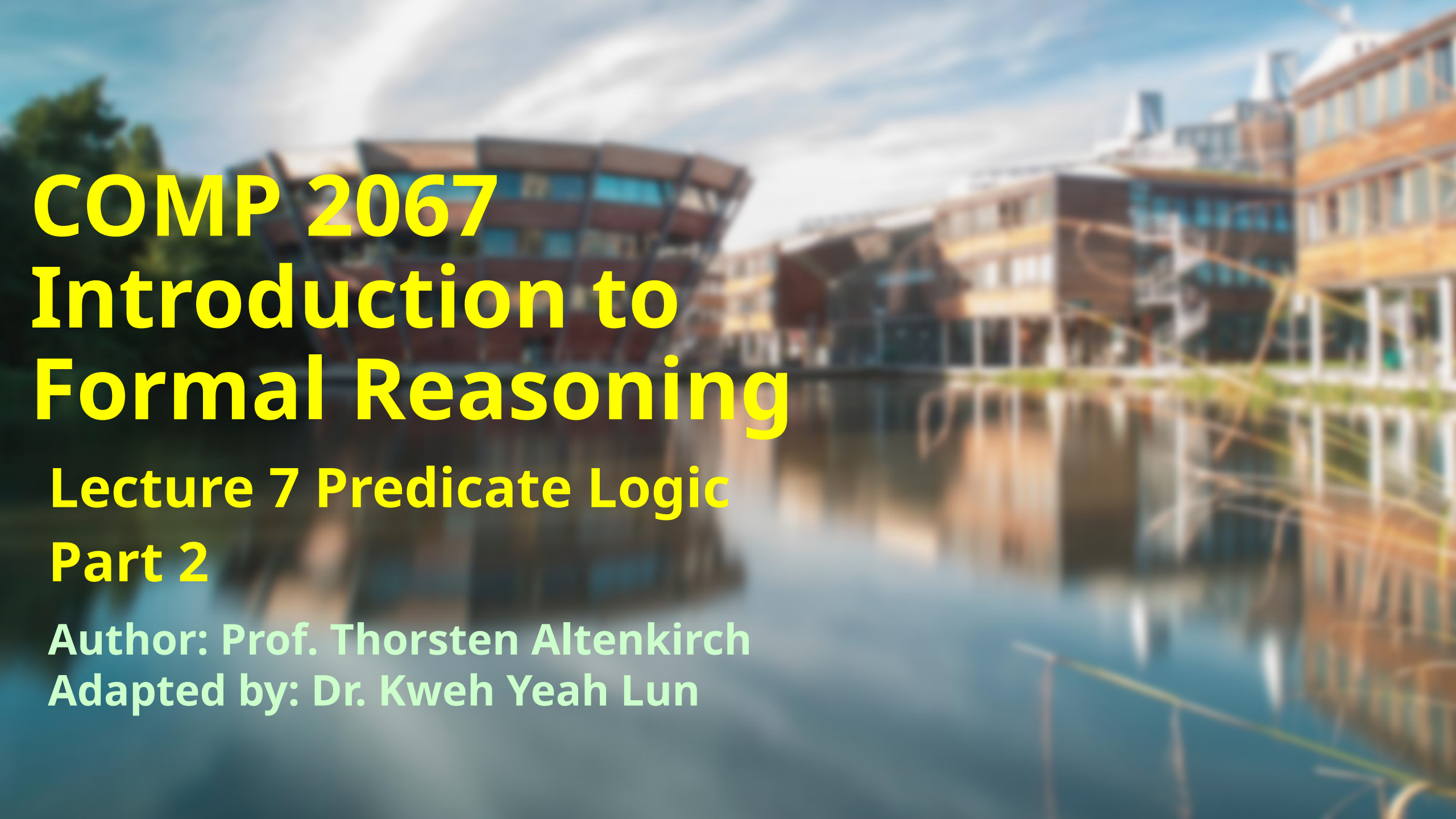

COMP 2067 Introduction to Formal Reasoning

Lecture 7 Predicate Logic Part 2

Author: Prof. Thorsten Altenkirch
Adapted by: Dr. Kweh Yeah Lun

University of Nottingham
UK | CHINA | MALAYSIA

Learning Outcomes

At the end of this lecture, you will be able to

- explain the currying equivalence in predicate logic and prove it using Lean
- understand and prove the property of an equivalence relation
- explain the classical predicate logic and prove laws that are associate to it

University of Nottingham
UK | CHINA | MALAYSIA

Currying Equivalence

- While currying in propositional logic had the form

  $P \wedge Q \to R \leftrightarrow P \to Q \to R$

- A conjunction is turned into an implication.
- Currying for predicate logic has the form

  $(\exists\ x : A, QQ\ x) \to R \leftrightarrow (\forall\ x : A , QQ\ x \to R)$

- An existential is turned into a universal quantifier.

- For the intuition, QQ x to mean x is clever and R means the professor is happy.
- Hence the equivalence is:

If there is a student who is clever then the professor is happy is equivalent to saying if any student is clever then the professor is happy.

- Example:

```
theorem curry_pred : (∃ x : A, PP x) → R  ↔ (∀ x : A , PP x → R)  :=
begin
  constructor,
  assume ppr a p,
  apply ppr,
  existsi a,
  exact p,
  assume ppr pp,
  cases pp with a p,
  apply ppr,
  exact p,
end
```

Try it!

University of Nottingham
UK | CHINA | MALAYSIA

Equality

- Given a b : A we can construct **a = b : Prop** expressing that **a and b are equal.**
- Can prove that everything is equal to itself using the tactic **reflexivity**

```
example : ∀ x : A, x=x :=
begin
  assume a,
  reflexivity,
end
```

Try it!

- Tactic **reflexivity**
  - **prove goals of the form x = x**, where x is any expression.
- This tactic is based on the **reflexive property of equality**, which states that **any term is equal to itself.**

- If an equality h : a = b is assumed, then can use it to rewrite a into b in the goal.
- That is if the goal is PP a, then rewrite h and this changes the goal into PP b.

Try it!

```
example:  ∀ x y : A, x=y → PP y → PP x :=
begin
  assume x y eq p,
  rewrite eq,
  exact p,
end
```

- Tactic **rewrite**
  - rewrite an expression in a goal or hypothesis using an equality lemma or hypothesis.
- It allows you to replace occurrences of a specified term with another term that is equal to it.

- The equality can be used in the other direction, that is to replace b by a.
- In this case: **rewrite← h.**

```
example:  ∀ x y : A, x=y → PP x → PP y :=
begin
  assume x y eq p,
  rewrite← eq,
  exact p,
end
```

Try it!

University of Nottingham
UK | CHINA | MALAYSIA

- Examples:

```
example : ∀ x y : A, x = y → PP x → PP y :=
begin
   assume x y eq hx,
   rewrite← eq,
   exact hx,
end

example : ∀ x y : A, x = y → PP x → PP y :=
begin
  assume x y eq hx,
  rewrite eq at hx,
  exact hx,
end
```

Try it!

- **rewrite eq at hx:**
  - rewrites the assumption hx using the equality eq.
  - This means that all occurrences of x in hx are replaced by y, transforming hx into a proof of PP y.

University of Nottingham
UK | CHINA | MALAYSIA

Equivalence relation

- **Equality is an equivalence relation**
- This means that it is
  - **reflexive (∀ x : A, x=x),**
  - **symmetric (∀ x y : A, x=y → y=x)**
  - **transitive (∀ x y z : A, x=y → y=z → x=z)**

- Symmetric property

```
theorem sym_eq : ∀ x y : A, x=y → y=x :=
begin
  assume x y p,
  rewrite p,
end
```

Try it!

- After assume, the goal

```
x y : A,
p : x = y
⊢ y = x
```

- After rewrite p, the goal would be expected to be

```
x y : A,
p : x = y
⊢ y = y
```

- Actually rewrite automatically applies reflexivity if possible, hence it is done.

- Transitivity property

Try it!

```
theorem trans_eq : ∀ x y z : A, x=y → y=z → x=z :=
begin
  assume x y z xy yz,
  rewrite xy,
  exact yz,
end
```

- Sometimes you want to use an equality not to rewrite the goal but to rewrite another assumption.
- Another proof of transitivi

```
theorem trans_eq : ∀ x y z : A, x=y → y=z → x=z :=
begin
  assume x y z xy yz,
  rewrite<- xy at yz,
  exact yz,
end
```

Try it!

- rewrite xy at yz: using xy to rewrite yz.
- The same works for rewrite←.

University of Nottingham
UK | CHINA | MALAYSIA

- Actually, Lean already has built-in tactics to deal with symmetry and transitivity which are often easier to use:

Try it!

```
example : ∀ x y : A, x=y → y=x :=
begin
  assume x y p,
  symmetry,
  exact p
end

example : ∀ x y z : A, x=y → y=z → x=z :=
begin
  assume x y z xy yz,
  transitivity,
  exact xy,
  exact yz,
end
```

- After transitivity, the goals looks a bit strange:
- The ?m_1 is a placeholder since Lean cannot figure out at this point what we are going to use as the intermediate term.
- Can just proceed, because as soon as the line exact xy is reached, Lean can figure out that ?m_1 is this left with:

```
A : Type,
x y z : A,
xy : x = y,
yz : y = z
⊢ y = z
```

which is the 2nd goal with ?m_1 instantiated

```
2 goals
A : Type,
x y z : A,
xy : x = y,
yz : y = z
⊢ x = ?m_1

A : Type,
x y z : A,
xy : x = y,
yz : y = z
⊢ ?m_1 = z
```

- Example: not going to use transitivity in equational proofs but use a special format for equational proofs indicated by **calc**:

```
example : ∀ x y z : A, x=y → y=z → x=z :=
begin
  assume x y z xy yz,
  calc
    x = y   : by exact xy
    ... = z : by exact yz,
end
```

Try it!

- After calc, prove a sequence of equalities where each step is using by followed by some tactics.
- Any subsequent line starts with ... which stands for the last expression of the previous line, in this case y.

- One more property from equality.
- Assume a function $f : A \to B$ and $x = y$ for some $x\ y : A$ then conclude that $f\ x = f\ y$.
- **Equality is a congruence.**
- Prove this easily using **rewrite**:

Try it!

```
theorem congr_arg : ∀ f : A → B , ∀ x y : A, x = y → f x = f y :=
begin
  assume f x y h,
  rewrite h,
end
```

University of Nottingham
UK | CHINA | MALAYSIA

Classical Predicate Logic

- Can use classical logic in predicate logic even though the explanation using truth tables doesn’t work anymore.
- There are predicate logic counterparts of the de Morgan laws which now say that you can move negation through a quantifier by negating the component and switching the quantifier.

- And again, one of them is provable using intuitionistic:

[Try it!]

```
theorem dm1_pred : ¬ (∃ x : A, PP x) ↔ ∀ x : A, ¬ PP x :=
begin
  constructor,
  assume h x p,
  apply h,
  existsi x,
  exact p,
  assume h p,
  cases p with a pa,
  apply h,
  exact pa,
end
```

- While the other direction again needs classical logic:

```
theorem dm2_pred : ¬ (∀ x : A, PP x) ↔ ∃ x : A, ¬ PP x :=
begin
  constructor,
  assume h,
  apply raa,
  assume ne,
  apply h,
  assume a,
  apply raa,
  assume np,
  apply ne,
  existsi a,
  exact np,
  assume h na,
  cases h with a np,
  apply np,
  apply na,
end
```

Try it!

- Example on em

```
variables A : Type
variables PP : A → A → Prop

open classical

example: ∀ x y : A, PP x y ∨ ¬PP x y :=
begin
  assume x y,
  cases em (PP x y) with eq neq,
  left,
  exact eq,
  right,
  exact neq,
end
```

Try it

- Tactic cases em:
  - Previously, cases em P with p np
  - In general: cases em HP with Hp Hnp
  - The cases em tactic is used to split the proof into two cases:
    - which is HP is true (Hp) or HP is false (Hnp).

University of Nottingham
UK | CHINA | MALAYSIA

Summary of tactics

	How to prove ?	How to use?
∀	`assume h`	`apply h`
∃	`existsi a`	`cases h with x p`
=	`reflexivity`	`rewrite h` `rewrite← h`

University of Nottingham
UK | CHINA | MALAYSIA

Typing symbol in Lean

- \not: ¬
- \forall: ∀
- \exists: ∃
- \iff: ↔
- \rightarrow: →
- \leftarrow: ←

University of Nottingham
UK | CHINA | MALAYSIA

- Give yourself a big clap.
- Survived once again? Great! ⏪
- Any question?

University of Nottingham
UK | CHINA | MALAYSIA

Next lecture

- Boolean

University of Nottingham
UK | CHINA | MALAYSIA

"The best way to predict the future is to invent it."

– Sam Altman

Hope you all enjoy this lecture!